

**Synopsis and Report
For**

College Event Management System

Database Management System (UCS310)

Submitted by:

102303347 Arpit Agarwal

102303346 Arun Mahajan

102303361 Arnav

BE Second Year

Batch: 2C25

Submitted to-

Dr. Manisha Panjeta



Computer Science and Engineering Department

TIET, Patiala

Introduction & Objective

The **updated Event Management System** is designed to provide a comprehensive framework for planning, organizing, and analyzing events of various categories, such as **hackathons, seminars, and workshops**. The **Entity-Relationship (ER) diagram** reflects a robust structure where each event is clearly defined, participants are categorized (e.g., faculty, students), and feedback loops are incorporated to measure the success of each event. By consolidating all relevant data—event details, registration information, and feedback—into a unified database, organizers can streamline their workflows and maintain high data integrity.

Key Aims

1. **Unified Event Data Management:** All event-related information (dates, venues, categories, organizers) is contained within a single system, reducing fragmentation.
2. **Participant Tracking:** Differentiation between student and faculty participants provides granular insights and allows for role-specific access and privileges.
3. **Feedback Integration:** Built-in feedback collection, including ratings and comments, helps organizers gauge participant satisfaction and refine future events.

Problem Statement

Coordinating large-scale or recurring events often involves many moving parts—handling registrations, generating tickets, assigning resources, and gathering feedback. Without a well-structured system, these tasks become prone to miscommunication, data duplication, and oversight. The **Event Management System** addresses these challenges by offering:

- A clear separation of participant types (**student or faculty**), ensuring accurate data entry and role-based management.
 - **Event categorization** (hackathon, seminar, workshop) to reflect distinct requirements like prize money, speaker details, or instructor involvement.
 - A **centralized feedback mechanism**, linking participants' comments and ratings to specific events, thus closing the loop for continuous improvement.
-

Scope of the Project

This project provides an **end-to-end solution** for all primary stakeholders in the event organization process. Drawing on the detailed ER diagram, the system integrates the following critical components:

1. **Person Entity:**

- Student and Faculty sub-entities capture different user attributes like **department, roll numbers, and branch**. This allows for targeted communication and role-specific permissions.

2. **Event Entity:**

- Each event has essential attributes (**event ID, name, description, category, date, time, venue**) and can also be further specialized (**hackathon, seminar, workshop**), each with its own set of details (e.g., **prize money** for hackathons).

3. **Registration Entity:**

- Tracks all participation details, including **registration dates, times, ticket IDs**, and extra notes, ensuring a straightforward process for enlisting participants.

4. **Feedback Entity:**

- Records participant perspectives via **feedback IDs, timestamps, ratings, and comments**, thereby providing organizers with insights into the event's performance.

5. **Organized By Relationship:**

- Links each event with its **organizing club**, including attributes such as **club ID and category**, giving structure and accountability to the event planning process.

The system's scope is broad enough to accommodate **multiple event types and participant categories**, yet modular enough to adapt to new or evolving requirements.

Methodology & Technologies Used

1. Entity-Relationship (ER) Modeling:

- The newly refined ER diagram serves as the project's blueprint, highlighting the structure and relationships of each entity (Person, Event, Registration, Feedback, Club, etc.). By distinguishing Student and Faculty sub-types under a Person entity, the design enforces a clear separation of user roles.

2. Relational Database Design:

- The ER model is converted into a fully normalized schema with primary keys, foreign keys, and well-defined constraints. This approach minimizes redundancy and maintains data integrity.

3. SQL and Database Management:

- SQL (Structured Query Language) is employed for creating tables, inserting and retrieving event data, and enforcing relational integrity through joins and constraints.

4. Express.js, React and Node.js :

- To build an interactive and dynamic user interface for event participants and to handle server-side logic, API routing, authentication, and data processing.

Expected Outcome

Upon completion, the Event Management System is expected to offer:

1. Efficient Data Handling:

- Seamlessly storing and managing large volumes of data for both events and participants, reducing manual work and error margins.

2. Structured Participant Management:

- Clear distinctions between student and faculty participants, allowing role-specific communications, event access, and data analytics.

3. Enhanced Event Oversight:

- The system ties together all stages of event planning, from registration to feedback. Organizers can quickly track attendance, collect participant impressions, and manage diverse event categories.

4. Actionable Insights:

- Through integrated feedback, event coordinators and clubs gain real-time visibility into participant satisfaction, enabling data-driven improvements for future events.

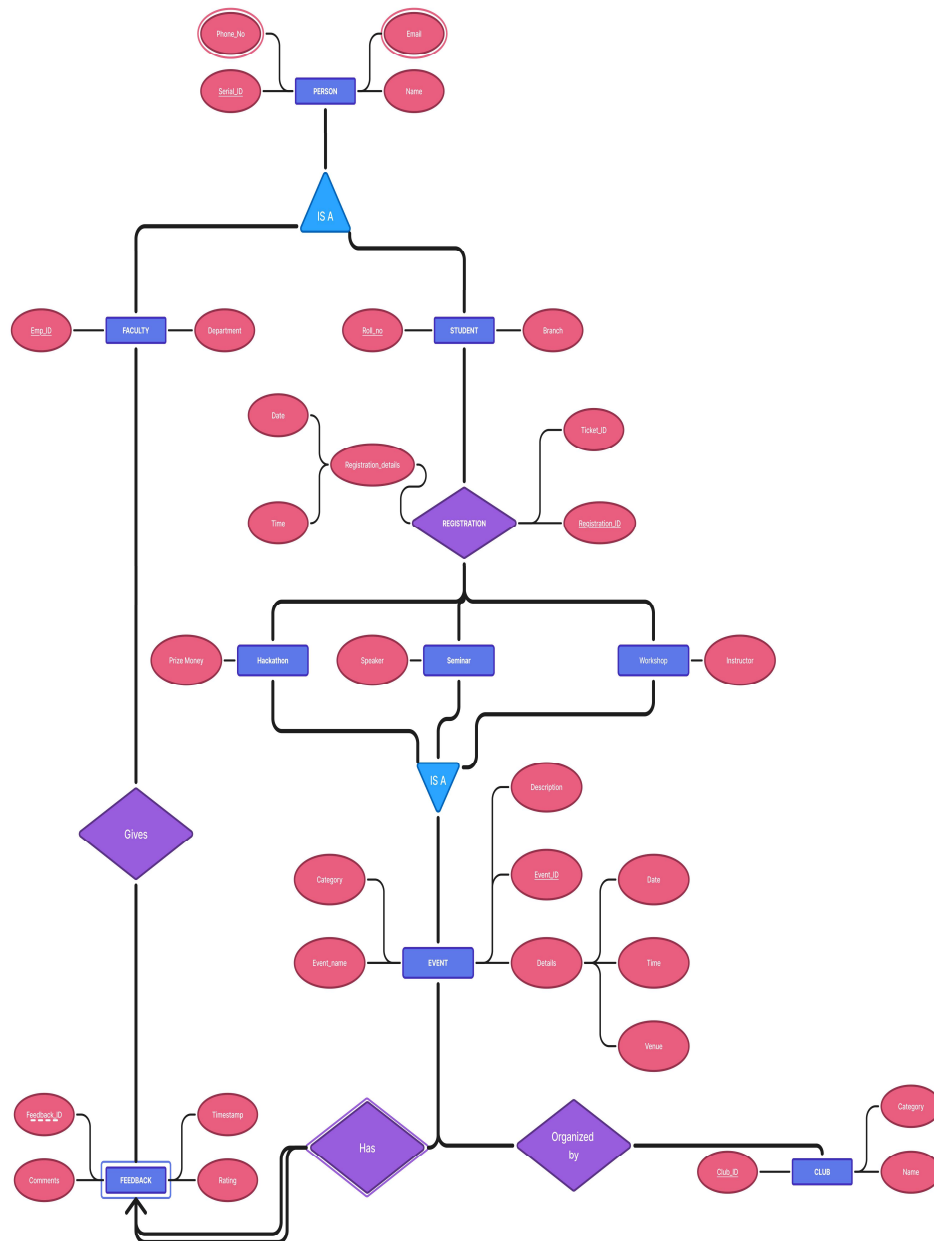
5. Scalable Foundation:

- The design caters to a variety of event formats and participant types, providing room for expansion into more advanced features (e.g., online payment gateways, automated scheduling, or third-party integrations).

Conclusion

Integrating a comprehensive set of entities—from participant profiles and event details to organizational and financial data—the College Event Management System offers a state-of-the-art solution to the multifaceted challenges of managing college events. By unifying these elements within a single, scalable platform, the system not only enhances operational efficiency but also fosters a proactive and data-driven approach to event planning. This holistic approach promises to transform how college events are conceived, managed, and evaluated, ultimately contributing to a more organized and engaging campus life.

ER Diagram



Entity Sets & Attributes

- **Person :**

1. Person (Serial_ID, Name) –

Here Serial_ID is the best choice for the primary key in PERSON because it is universal across different person types and unique and system-generated.

2. P1 (Serial_ID*, Phone_No) –

Here Serial_ID is foreign key because it accurately link a person to multiple records (like registrations or feedback). And Serial_ID and Phone_No together here act as a primary key.

3. P2 (Serial_ID*, Email) –

Here Serial_ID is foreign key because it accurately link a person to multiple records (like registrations or feedback). And Serial_ID and Phone_No together here act as a primary key.

- **Faculty :**

Faculty (Emp_ID, Serial_ID*, Department) –

In the faculty table, Emp_ID is the primary key because it uniquely identifies each faculty member, while Serial_ID is a foreign key linking to the PERSON table to inherit shared attributes like name and contact details.

- **Student :**

1. Student (Roll_No, Serial_ID*, Branch) –

In the student table, Roll_No is the primary key as it uniquely identifies each student, while Serial_ID is a foreign key referencing the PERSON table to access common personal details like name, phone number, and email.

2. Registration (Event_ID*, Roll_No*, Registration_ID, Date, Time, Ticket_ID) –

In the registration table, Event_ID, Registration_ID and Roll_No together form a composite primary key to uniquely identify each student's registration for a specific event, while Registration_ID serves as a key attribute, and the additional attributes like Date, Time, and Ticket_ID store registration details. Here Event_ID and Roll_No are foreign keys.

- **Hackathon:**

Hackathon (Event_ID*, Prize Money) -

In the hackathon table, Event_ID is the primary key as it uniquely identifies each hackathon event and also acts as a foreign key referencing the EVENT table, while Prize Money is a specific attribute related only to hackathon-type events.

- Seminar :

Seminar (Event_ID*, Speaker) -

In the seminar table, Event_ID is the primary key as it uniquely identifies each seminar event and also acts as a foreign key referencing the EVENT table, while speaker is a specific attribute related only to Seminars .

- Workshop :

Workshop (Event_ID*, Instructor) -

In the workshop table, Event_ID is the primary key as it uniquely identifies each workshop and also acts as a foreign key referencing the EVENT table, while Instructor is a specific attribute related only to Workshop .

- Event :

1. Event (Event_ID, Category, Event_name, Description, Date, Time, Venue) -

In the event table, Event_ID is the primary key as it uniquely identifies each event, while the other attributes like Category, Event_name, Description, Date, Time, and Venue provide detailed information about the event.

2. Orgaized_by (Club_ID*,Event_ID*) –

In the organized_by table, Club_ID and Event_ID together form a composite primary key representing the many-to-many relationship between clubs and events, where each event can be organized by multiple clubs and each club can organize multiple events; both also act as foreign keys referencing the CLUB and EVENT tables respectively.

- Club:

Club (Club_ID,category,name) –

In the Club table, Club_ID is the primary key as it uniquely identifies each club, while category and name store descriptive information about the club's type and title.

- Feedback:

1. Feedback (Feedback_ID,Event_ID*,timestamp,comments,rating) –

In the Feedback table, Feedback_ID and Event_ID together form a composite primary key to uniquely identify each feedback entry given for a specific event and Event_ID act as foreign key refrencing EVENT table .

2. Gives (Emp_ID*,Feedback_ID*) –

In the Gives table, Emp_ID and Feedback_ID together form a composite primary key representing the relationship between faculty and the feedback they provide, with both also acting as foreign keys referencing the FACULTY and FEEDBACK tables respectively.

RELATIONSHIPS, CARDINALITY & PARTICIPATION

1. PERSON — IS A — STUDENT / FACULTY

- **Cardinality:** One-to-One
Each person is either a student or a faculty member, not both.
- **Participation:** Total participation from STUDENT and FACULTY
Every student and faculty member must be associated with an existing person in the PERSON entity.

2. PERSON — REGISTRATION — EVENT

- **Cardinality:** Many-to-Many
A person can register for multiple events, and each event can have multiple participants.
- **Participation:** Partial
Not every person is required to register for an event, and not every event must have participants at all times.

3. EVENT — IS A — HACKATHON / SEMINAR / WORKSHOP

- **Cardinality:** One-to-One
Each event is exactly one of the subtypes—hackathon, seminar, or workshop.
- **Participation:** Total participation from subtypes
Every Hackathon, Seminar, or Workshop must be a subtype of EVENT.

4. EVENT — ORGANIZED_BY — CLUB

- **Cardinality:** Many-to-Many
An event may be organized by multiple clubs, and a club can organize multiple events.
- **Participation:** Partial
Not all events must be associated with a club, and not all clubs need to organize an event.

5. FACULTY — GIVES — FEEDBACK

- **Cardinality:** One-to-Many
A faculty member may give multiple feedback entries, but each feedback is tied to a single faculty.
- **Participation:** Partial
Not every faculty member is required to provide feedback.

6. EVENT — HAS — FEEDBACK

- **Cardinality:** One-to-Many
An event can receive multiple feedback entries, but each feedback entry belongs to one event.

- **Participation:** Partial
Feedback is optional for events and may not always be available.

Normalization

1. First Normal Form (1NF): Achieved

- **Atomic Attributes:** All attributes (like Name, Email, Department, etc.) store atomic (indivisible) values.
- **No Repeating Groups:** There are no sets of repeating fields for a single entity.
- **Unique Identifiers:** Each entity has a unique identifier (e.g., Serial_ID, Emp_ID, Ticket_ID, etc.).

2. Second Normal Form (2NF): Achieved

- **Full Functional Dependency:** All non-key attributes in composite keys (like in REGISTRATION) depend on the whole primary key and not just part of it.
- For example, REGISTRATION relates to a unique combination of Person and Event (though not explicitly shown as a composite key in the diagram, the relationship implies it). Attributes like Registration details, Time and Date are fully dependent on the registration entry.
- **Partial Dependencies Removed:** There's a clear separation of student and faculty into their own sub-entities, so their respective attributes (e.g., Branch for students, Department for faculty) do not depend on partial keys.

3. Second Normal Form (2NF): Achieved

- The system removes transitive dependencies—attributes don't depend on other non-key attributes.
- For instance, an event's venue or time depends only on the Event_ID, not on other details like the event name.
- Similarly, feedback ratings and comments are tied directly to a unique feedback ID or registration, not indirectly through the person or event.